

Lista de Exercícios AC2 - Vale 0,18

1. Sistema de Instrumentos Musicais

Crie uma classe `Instrumento` com os atributos privados `nome` e `marca`. Crie getters para acessar essas informações.

Depois crie duas subclasses:

- `Violao`
- `Piano`

Cada classe deverá sobrescrever:

Python

```
def tocar(self):  
    pass
```

O violão deve imprimir algo como:

None

```
0 violão está sendo tocado.
```

E o piano:

None

```
0 piano está sendo tocado.
```

Crie também uma classe `Musico` com o atributo privado `nome`.

Cada instrumento deve possuir um músico responsável.

Exemplo:

- Violao **TEM UM** Musico
- Piano **TEM UM** Musico

No final:

Python

```
instrumentos = [  
    Violao(...),  
    Piano(...)  
]  
  
for instrumento in instrumentos:  
    instrumento.tocar()
```

2. Sistema de Produtos

Crie uma classe **Produto** com os atributos privados:

- **nome**
- **preco**

Crie getters para os dois atributos.

Depois crie:

- **ProdutoFisico**
- **ProdutoDigital**

Cada classe deverá implementar:

Python

```
def entregar(self):  
    pass
```

Exemplos:

None

0 produto físico será enviado pelos Correios.

None

0 produto digital será enviado por download.

Crie também uma classe:

Fornecedor

com atributo privado:

nome

Cada produto deve possuir um fornecedor.

No final:

```
Python
produtos = [
    ProdutoFisico(...),
    ProdutoDigital(...)
]

for produto in produtos:
    produto.entregar()
```

3. Sistema de Restaurantes

Crie uma classe `Prato` com os atributos privados:

- `nome`
- `preco`

Depois crie duas subclasses:

- `PratoPrincipal`
- `Sobremesa`

As duas devem implementar:

```
Python
def preparar(self):
    pass
```

Exemplos:

None

Preparando o prato principal.

None

Preparando a sobremesa.

Crie também uma classe:

Cozinheiro

com o atributo privado `nome`.

Cada prato deve possuir um cozinheiro responsável.

Depois:

Python

```
pratos = [  
    PratoPrincipal(...),  
    Sobremesa(...)  
]  
  
for prato in pratos:  
    prato.preparar()
```

4. Sistema de Jogos

Crie uma classe `Personagem` com os atributos privados:

- `nome`
- `vida`

Crie getters para os atributos.

Depois crie:

- `Guerreiro`
- `Mago`

Ambos devem implementar:

Python

```
def atacar(self):  
    pass
```

Exemplo:

None

0 guerreiro atacou com uma espada.

None

0 mago lançou uma magia.

Crie também uma classe:

Arma

com atributo privado:

nome

Cada personagem deve possuir uma arma.

Depois:

Python

```
personagens = [  
    Guerreiro(...),  
    Mago(...)  
]  
  
for personagem in personagens:  
    personagem.atacar()
```

5. Sistema de Transporte Público

Crie uma classe `Transporte` com os atributos privados:

- `linha`
- `capacidade`

Depois crie:

- `Onibus`
- `Metro`

Ambos devem implementar:

Python

```
def transportar(self):  
    pass
```

Exemplo:

None

0 ônibus está transportando passageiros.

None

0 metrô está transportando passageiros.

Crie também uma classe:

`Motorista`

com atributo privado `nome`.

O ônibus deve possuir um motorista.

Para o metrô, crie uma classe:

`Condutor`

com atributo privado `nome`.

No final:

Python

```
transportes = [  
    Onibus(...),  
    Metro(...)  
]  
  
for transporte in transportes:  
    transporte.transportar()
```

6. Sistema de Livros

Crie uma classe **Publicacao** com os atributos privados:

- **titulo**
- **ano**

Depois crie:

- **Livro**
- **Revista**

Ambas devem implementar:

Python

```
def exibir_informacoes(self):  
    pass
```

Exemplos:

None

Livro: Programação em Python

None

Revista: Tecnologia Hoje

Crie também uma classe:

Editora

com atributo privado `nome`.

Cada publicação deve possuir uma editora.

Depois:

```
Python
publicacoes = [
    Livro(...),
    Revista(...)
]

for publicacao in publicacoes:
    publicacao.exibir_informacoes()
```

7. Sistema de Hospedagem

Crie uma classe `Hospedagem` com os atributos privados:

- `nome`
- `valor_diaria`

Depois crie:

- `Hotel`
- `Pousada`

Ambos devem implementar:

```
Python
def reservar(self):
    pass
```

Exemplos:

None

Reserva realizada no hotel.

None

Reserva realizada na pousada.

Crie também uma classe:

Endereco

com atributos privados:

- cidade
- estado

Tanto `Hotel` quanto `Pousada` devem possuir um endereço.

Depois:

Python

```
hospedagens = [  
    Hotel(...),  
    Pousada(...)  
]  
  
for hospedagem in hospedagens:  
    hospedagem.reservar()
```

8. Sistema de Entregas

Crie uma classe `Entrega` com os atributos privados:

- destino
- peso

Depois crie:

- `EntregaMoto`
- `EntregaCaminhao`

As duas devem implementar:

Python

```
def realizar_entrega(self):  
    pass
```

Exemplos:

None

Entrega sendo realizada de moto.

None

Entrega sendo realizada de caminhão.

Crie também uma classe:

Entregador

com atributo privado `nome`.

Cada entrega deve possuir um entregador responsável.

No final:

Python

```
entregas = [  
    EntregaMoto(...),  
    EntregaCaminhao(...)  
]  
  
for entrega in entregas:  
    entrega.realizar_entrega()
```

9. Sistema de Cursos Online

Crie uma classe `CursoOnline` com os atributos privados:

- `titulo`
- `carga_horaria`

Depois crie:

- `CursoProgramacao`
- `CursoDesign`

Ambos deverão implementar:

```
Python
def iniciar_aula(self):
    pass
```

Exemplo:

```
None
Iniciando aula de programação.
```

```
None
Iniciando aula de design.
```

Crie também uma classe:

`Instrutor`

com atributo privado:

`nome`

Cada curso deverá possuir um instrutor.

Depois:

```
Python
cursos = [
```

```
        CursoProgramacao(...),  
        CursoDesign(...)  
]  
  
for curso in cursos:  
    curso.iniciar_aula()
```

10. Sistema de Serviços

Crie uma classe `Servico` com os atributos privados:

- `descricao`
- `valor`

Depois crie:

- `ServicoLimpeza`
- `ServicoManutencao`

Ambos deverão implementar:

```
Python  
def executar(self):  
    pass
```

Exemplos:

```
None  
Executando serviço de limpeza.
```

```
None  
Executando serviço de manutenção.
```

Crie também uma classe:

`Profissional`

com atributo privado:

nome

Cada serviço deverá possuir um profissional responsável.

No final:

Python

```
servicos = [  
    ServicoLimpeza(...),  
    ServicoManutencao(...)  
]  
  
for servico in servicos:  
    servico.executar()
```

Esses 10 novos exercícios continuam exatamente a mesma lógica:

Herança: *Guerreiro* é um *Personagem*.

Composição: *Guerreiro* tem uma *Arma*.

Encapsulamento: atributos como `__nome`, `__vida` e `__preco` ficam protegidos.

Polimorfismo: diferentes objetos respondem ao mesmo método, como `atacar()`, `entregar()`, `preparar()` ou `executar()`, mas cada um com comportamento próprio.